EXPERIMENT 9: FORWARD CHAINING

AIM

To demonstrate forward chaining in Prolog by deriving new facts from initially known facts and rules.

PROCEDURE

1. Define the initial facts in the knowledge base.

2. Apply rules whose conditions match the known facts.

3. Run the start query and observe the inferred result.

CODE

1. Simple Inference

```
% Dynamic knowledge base
:- dynamic known/1.

% Initial knowledge
start :-
    assert(known(a)),
    assert(known(b)),
    infer.

% Rule
infer :-
    known(a), known(b), \+ known(c),
    assert(known(c)),
    write('Inferred c'), nl.

% Query
?- start.
```

OUTPUT

```
Inferred c
```

2. Fire Alert System

```
:- dynamic known/1.

% Rule to infer fire
check :-
    known(smoke), known(heat),
    assert(known(fire)),
    write('Fire Alert!'), nl.

% Start rule
start :-
    assert(known(smoke)),
    assert(known(heat)),
    check.

% Query
?- start.
```

OUTPUT

```
Fire Alert!
```

3. Water Purity Detection

```
:- dynamic known/1.

% Rule to infer pollution
infer_pollution :-
    known(cloudy_water), known(bad_smell),
    assert(known(polluted)),
    write('Water is polluted'), nl.

start :-
    assert(known(cloudy_water)),
    assert(known(bad_smell)),
    infer_pollution.

% Query
?- start.
```

OUTPUT

```
Water is polluted
```

RESULT

Thus, forward chaining was demonstrated using Prolog rules to infer a new fact, raise a fire alert, and identify polluted water.